

# LLM Exam

## C1. Deep Learning

The perceptron is the simplest unit in neural networks. It takes multiple input features, assigns weights to them, computes a weighted sum, and applies an activation function. Without an activation function, the perceptron behaves similarly to linear regression, while the application of a sigmoid activation function makes it resemble logistic regression.

### Activation Functions

- Enforce constraints on the perceptron's output, e.g., the sigmoid function limits values between 0 and 1.
- Introduce non-linearity, allowing the network to learn more complex patterns.
- Common activation functions: ReLU, Sigmoid, Tanh, Softmax, and GeLU.

When multiple perceptrons are stacked together with activation functions applied between layers, a more powerful model is created, known as a **Multi-Layer Perceptron (MLP)**. According to the Universal Approximation Theorem, a neural network with at least one hidden layer can approximate any continuous function. Deeper and narrower networks are typically used for more complex tasks.

In **classification** and **regression**, different approaches are used. For binary classification, the sigmoid function is employed to output probabilities. Multi-class classification uses the softmax function to obtain probability distributions across multiple classes. Regression typically uses a linear output without activation functions.

The network learns by **minimizing a loss function**. The optimization process uses **Gradient Descent**, which updates weights in the direction that reduces the loss. **Backpropagation** is used to compute how much each weight contributes to the error and applies the chain rule to propagate errors from the output back to the input layer.

### Loss Functions

- **Regression**: Uses Mean Squared Error (MSE) or Mean Absolute Error (MAE).

- **Binary Classification:** Uses Binary Cross-Entropy (BCE).
- **Multi-class Classification:** Uses Cross-Entropy Loss.

### Optimization Challenges

- Weight initialization affects convergence.
  - Local minima can slow down training.
  - Solutions: momentum, adaptive learning rates, and learning rate scheduling.
- 

## C2. Word Embeddings

Word embeddings are dense vector representations of words, as opposed to sparse representations like one-hot encoding. Each word is mapped to a high-dimensional vector (e.g., 300 dimensions) to capture different aspects of meaning. Word embeddings preserve **semantic similarity**, meaning words with similar meanings have similar vector representations.

### One-Hot Encoding and Its Limitations

One-hot encoding represents words as sparse vectors where only one dimension is active. However, it has several limitations:

- **Scalability:** With a large vocabulary (e.g., 50,000 words), the vectors become extremely high-dimensional.
- **Lack of semantic similarity:** All word vectors are orthogonal, meaning they don't preserve meaning relationships.

### Distributed Representations

Unlike one-hot encoding,

**distributed representations** spread word information across multiple dimensions. These representations are learned through tasks such as predicting missing words in a sentence.

### Learning Word Embeddings

The learning process for word embeddings typically follows these steps:

1. Assign random vectors to words.
2. Adjust word vectors based on context (neighboring words).

3. Apply gradient descent to refine embeddings.
4. Repeat across millions of sentences, ensuring that words appearing in similar contexts have similar embeddings.

### Matrix Representation of Word Embeddings

Word vectors are stored in matrices:

- **Input matrix:** Contains vectors for context words.
- **Output matrix:** Contains vectors for predicted words.  
The model uses the **dot product** to measure similarity and applies a **softmax** function to compute probabilities.

### Neural Language Models and Word2Vec

Word2Vec is a popular model for learning word embeddings. It has two main approaches:

- **Continuous Bag of Words (CBOW):** Predicts a missing word given its context.
- **Skip-Gram:** Predicts surrounding words given a central word.  
To improve efficiency, Word2Vec uses:
- **Hierarchical softmax:** Speeds up probability computation.
- **Negative sampling:** Reduces computational cost.

### Limitations of Word2Vec

- **Out-of-Vocabulary (OOV) Words:** Word2Vec cannot handle words not present in the training data.
- **Lack of Context Awareness:** A word's meaning is fixed, even if it changes in different contexts (e.g., "bat" in sports vs. animals).

### FastText: An Improvement Over Word2Vec

FastText addresses the OOV problem by representing words as **subword n-grams** (e.g., "where" → ["<wh", "whe", "her", "ere", "re>"]). This allows it to generate embeddings for unseen words by combining their subword vectors.

### Visualizing Word Embeddings

- **Semantic Clustering:** Similar words group together in a lower-dimensional space (e.g., mammals vs. birds).

- **Relationships Between Words:** Vector operations can model relationships (e.g., "King – Man + Woman  $\approx$  Queen").
  - **Geographical Insights:** Country-capital relationships can be learned as vector transformations (e.g., "Spain + (Germany – Berlin)  $\approx$  Madrid").
- 

## C3. Recurrent Neural Networks

### Limitations of Fully Connected Neural Networks (FCNNs)

Fully Connected Neural Networks (FCNNs) are designed to handle fixed-size inputs and outputs, which makes them unsuitable for tasks involving variable-length sequences. They require a unique set of weights for each input, which leads to inefficiency when learning repeated patterns. Additionally, their fixed output size makes them problematic for tasks like object detection or sequence generation.

### What Are Recurrent Neural Networks (RNNs)?

Recurrent Neural Networks (RNNs) are designed specifically for sequential data and can handle inputs and outputs of varying lengths. They process one element at a time, such as a word in a sentence or a character in a string, while maintaining an internal hidden state that summarizes past information to influence future predictions.

### RNN Architecture

At each time step, an RNN takes:

- The current input element.
  - The hidden state from the previous step.
- It then produces:
- An output for the current step.
  - A new hidden state to be used in the next step.

The same model and weights are reused across all time steps.

### Training RNNs: Backpropagation Through Time (BPTT)

RNNs are trained using

**Backpropagation Through Time (BPTT)**, where:

- The model is "unrolled" over the entire sequence.

- The loss function is computed for all time steps.
- Gradients from each step are accumulated and used to update the weights.

### Problems with RNNs

- **Vanishing and Exploding Gradients:** Gradients can either shrink to zero (vanishing gradient) or grow exponentially (exploding gradient) due to their multiplication at each step.
- **Long-Term Dependency Issues:** RNNs struggle to retain information from distant time steps, making tasks like next-word prediction in long sentences difficult.
- **Computational Inefficiency:** RNNs cannot be parallelized, as each step depends on the previous one.

### Gated RNNs: LSTMs and GRUs

Gated RNNs, such as

**LSTMs (Long Short-Term Memory)** and **GRUs (Gated Recurrent Units)**, address some of the limitations of vanilla RNNs by introducing control mechanisms:

- **LSTM:** Uses gates to manage memory effectively:
  - **Forget Gate:** Decides what past information to discard.
  - **Input Gate:** Decides what new information to store.
  - **Output Gate:** Decides what part of the memory should be used in the output.
- **GRU:** A simpler version of LSTM with only two gates (Update and Reset).

### Sequence-to-Sequence (Seq2Seq) Models

Some tasks, such as machine translation, require mapping an input sequence to an output sequence. This is achieved using the

#### Encoder-Decoder Architecture:

- **Encoder:** Processes the input sequence and generates a context vector.
- **Decoder:** Uses this context vector to generate the output sequence, running until it predicts a special **End of Sequence (EOS)** token.

### Limitations of Gated RNNs

Despite improvements, LSTMs and GRUs still have:

- Difficulty handling very long sequences, as they still rely on a single context vector.
  - Vanishing/exploding gradients due to BPTT.
  - Lack of parallelization, making them slower than newer architectures like **Transformers**.
- 

## C4. Transformers

### Introduction to Transformers

Transformers are a sequence-to-sequence architecture introduced by Vaswani et al. (2017).

Unlike RNNs, they do not process inputs sequentially, enabling parallelization and eliminating long-term dependency issues.

They use an **attention mechanism** instead of recurrence to process sequences.

### Transformer Architecture

The original transformer model follows an **encoder-decoder structure**:

- **Encoder**: Processes the input sequence and generates hidden representations.
- **Decoder**: Uses the encoded representation to generate the output sequence step by step. The model applies **self-attention** to relate different parts of the sequence instead of using recurrence.

### Tokenization and Input Processing

Transformers operate on **tokens**, which can be:

- **Character-level**: Robust but inefficient.
- **Word-level**: Efficient but suffers from **out-of-vocabulary (OOV) issues**.
- **Subword-level** (e.g., **Byte-Pair Encoding, WordPiece**): A balance between the two approaches.

### Positional Encoding

Since transformers **do not process sequences sequentially**, they require a mechanism to represent token positions.

Positional encoding is added to input embeddings to inject order information, which can be:

- **Sinusoidal (fixed mathematical functions).**
- **Learned embeddings** (used in models like GPT).

## Attention Mechanism

The core idea of transformers is **self-attention**, which allows the model to focus on relevant tokens when making predictions.

Instead of treating all words equally, self-attention assigns different weights to each token based on its relevance.

Attention uses three components:

- **Query (Q):** Represents the token being processed.
- **Key (K):** Represents tokens to compare against the query.
- **Value (V):** The information to be retrieved.

The attention score is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

where  $d_k$  is a scaling factor to stabilize training.

## Multi-Head Attention

Instead of a single attention mechanism, transformers use **multiple attention heads** to focus on different relationships within the data.

Each attention head applies the self-attention mechanism separately, then their outputs are concatenated and transformed.

## Masked Self-Attention in Decoders

During training, the decoder must not peek at future tokens.

This is enforced by **masking future tokens**, setting their attention scores to negative infinity, preventing them from influencing the prediction.

## Encoder-Decoder Attention

The decoder needs information from the encoder to generate meaningful outputs.

It uses **cross-attention**, where:

- **Queries** come from the decoder.
- **Keys and values** come from the encoder's output. This allows the decoder to learn which parts of the input sequence are most relevant.

## Transformer Variants

- **Encoder-only models** (e.g., BERT):
  - Used for classification, named entity recognition, and question answering.
  - Uses **bidirectional attention**, meaning tokens attend to both past and future words.
- **Decoder-only models** (e.g., GPT):
  - Designed for text generation.
  - Uses **causal self-attention**, where each token attends only to past tokens.
- **Encoder-Decoder models** (e.g., T5, BART):
  - Suitable for tasks like translation, summarization, and text-to-text tasks.

## Text Generation and Sampling Strategies

When generating text, different sampling methods are used:

- **Greedy decoding**: Always picks the most probable next token (can be repetitive).
- **Beam search**: Expands multiple paths before selecting the best one.
- **Top-k sampling**: Chooses from the top-k most probable words.
- **Top-p (nucleus) sampling**: Samples from the smallest group of words whose cumulative probability exceeds a threshold.

- **Temperature scaling:** Adjusts randomness by modifying probability distributions.
- 

## C5a. LLM: History

IMAO useless 😏

---

## C5b. LLM: Metrics, Tasks, Benchmarks

Several key metrics are used to evaluate Large Language Models (LLMs), focusing on accuracy and reliability. **Geometric Mean** penalizes low values more than the arithmetic mean, while **Perplexity** measures model uncertainty when predicting the next token—low perplexity indicates high confidence, whereas high perplexity suggests uncertainty.

For assessing text generation quality, metrics like **BLEU** compare generated sequences with reference texts using n-grams and brevity penalties, whereas **ROUGE** is better suited for summarization as it focuses on recall. **METEOR** enhances BLEU by incorporating stemming, synonym matching, and fragmentation penalties. **BERTScore** compares token vectors of generated and reference sentences using similarity functions like cosine similarity. For binary evaluation, **Exact Match (EM)** assigns 1 only if the tokens match exactly. Additionally, **Ranking Metrics** (e.g., MRR, NDCG, precision@k) assess the ranking position of the correct token in a sorted probability list.

Different tasks test a model's capabilities. In **Text Completion**, the model predicts how a passage should continue. Benchmarks like **LAMBADA** evaluate its ability to guess a missing word when given full context, while datasets like **ROCStories**, **HellaSwag**, and **StoryCloze** require selecting the correct ending for a story.

**Question Answering (QA)** is divided into two types:

- **Open-book**, where the model can search for answers within a given text.
- **Closed-book**, which assesses the model's internal knowledge.

Common QA benchmarks include **SQuAD**, **TriviaQA**, **Natural Questions**, and **WebQuestions**.

For **translation**, datasets such as **WMT** are used. **Summarization** can be extractive or abstractive and is evaluated with datasets like **CNN/Daily Mail** and **PubMed Diabetes**.

In **Natural Language Inference (NLI)**, the model determines whether a hypothesis is entailed, contradicted, or neutral concerning a given premise. Benchmarks include **Stanford NLI** and **Multi-Genre NLI**. **Grammatical Acceptability** tests whether a sentence is grammatically correct, using datasets like **CoLA**.

Other common tasks include:

- **Text Classification**, such as sentiment analysis (**IMDb**, **Yelp**, **SST-2**) and topic classification (**AG News**, **20 Newsgroups**).
- **Mathematical Reasoning**, evaluated with datasets like **MATH** and **GSM8K**.
- **Commonsense Reasoning**, including ambiguity resolution (**Winograd Schema Challenge**, **Winogrande**) and causal reasoning (**COPA**, **ATOMIC**).
- **Synthetic Tasks**, which test specific capabilities like arithmetic, character removal, or word unscrambling.

Several benchmarks help standardize evaluation. **GLUE** (General Language Understanding Evaluation) provides a composite metric across multiple tasks, while **SuperGLUE** is a more challenging version. **MMLU** (Massive Multitask Language Understanding) is a QA benchmark covering 57 different subjects. One challenge in benchmarking is **LLM Contamination**, where public datasets leak into model training, making evaluations less reliable.

---

## C5c. LLM: Instruction Tuning & Model Alignment

Large Language Models (LLMs) are initially trained as **next-token predictors**, meaning they generate text based on statistical patterns rather than true understanding. However, AI assistants like ChatGPT must go beyond simple prediction, following instructions while ensuring responses are **helpful, honest, and harmless (HHH)**.

While LLMs exhibit **few-shot learning**, where they learn from limited examples, they struggle with **zero-shot tasks**, requiring them to generalize without prior examples. The **T5 Model** introduced instruction-based conditioning but had limitations in adapting to new tasks. The **T0 Model** improved this by fine-tuning on multiple QA datasets with diverse prompt templates, enhancing generalization.

Google's **FLAN (Finetuned Language Net)** scaled instruction tuning to larger models (up to 137B parameters), proving that this approach significantly boosts zero-shot performance. However, even larger models, despite better prediction capabilities, can still generate **untruthful, biased, or unhelpful** responses. This happens because traditional training methods fail in several ways:

- Metrics like **ROUGE** do not effectively measure response quality.
- Loss functions treat all errors equally, without prioritizing helpfulness.
- Models learn from both **high- and low-quality** data indiscriminately.

To address these issues, **Reinforcement Learning from Human Feedback (RLHF)** refines model behavior:

1. **Collect Human Feedback** – Humans rank AI-generated responses.
2. **Train a Reward Model (RM)** – An AI model learns to predict human preferences.
3. **Fine-Tune with RLHF** – The model is updated to generate preferred responses.

Despite its effectiveness, RLHF introduces challenges:

- **Human evaluation** is costly and inconsistent.
- **Reward models** may deviate from human intuition.
- **KL divergence regularization** is used to prevent excessive divergence from the model's original behavior.

A key milestone in AI alignment was **InstructGPT**, a fine-tuned version of GPT-3. Despite being much smaller (1.3B vs. GPT-3's 175B parameters), it achieved:

- **Stronger user preference** due to better instruction-following.
- **More truthful and less toxic** responses.

- **Improved generalization** to new tasks.

The modern AI alignment process follows a structured approach:

1. **Pretraining** on massive, noisy datasets to capture broad knowledge.
2. **Fine-tuning** on curated, high-quality datasets to improve instruction-following.
3. **RLHF** to align AI behavior with human expectations, ensuring safer interactions.

This methodology, pioneered by **InstructGPT**, laid the foundation for ChatGPT and other modern AI assistants.

---

## C5d. LLM: Efficient Fine-Tuning & Inference

LLMs like **GPT-3** and **FLAN** are capable of **few-shot** and **zero-shot learning**, but **fine-tuning** significantly improves their performance, often allowing smaller fine-tuned models to outperform much larger ones using in-context learning. However, **full fine-tuning** is computationally expensive, leading to the development of more **efficient techniques**.

### Fine-Tuning Techniques

- **Full Fine-Tuning**: Updates all model weights, achieving near from-scratch training performance but requiring high computational resources.
- **Feature-Based Transfer**: Freezes the base model and trains only a new head. This is efficient but struggles with diverse tasks.
- **Parameter-Efficient Fine-Tuning (PEFT)**: Updates only a subset of parameters, drastically reducing costs while maintaining strong performance.

### PEFT Techniques

- **BitFit**: Fine-tunes only **bias terms** (just 0.1% of BERT's parameters) with minimal cost but strong effectiveness.
- **Adapters**: Introduce small trainable layers between transformer layers, reducing trainable parameters **by 70x** compared to full fine-tuning.

- **LoRA (Low-Rank Adaptation):** Factorizes weight updates into two small matrices, requiring **140x fewer parameters** than full fine-tuning. Multiple tasks can share the same base model by swapping task-specific adapters.
  - **Prompt Tuning:** Instead of modifying model weights, it fine-tunes **only special tokens**, making it highly effective for instruction-following tasks.
- 

## Model Optimization for Efficient Inference

To deploy LLMs efficiently, techniques like **quantization** and **model distillation** optimize inference speed and memory usage while minimizing performance loss.

### 1. Quantization

Converts floating-point model weights to lower precision (e.g., **int8**, **float16**) to reduce memory and speed up inference.

- **Post-Training Quantization (PTQ):** Applied after training, with minor accuracy loss.
- **Quantization-Aware Training (QAT):** Applies quantization **during training**, maintaining better accuracy.
- **LLM.int8():** A hybrid approach that keeps **0.1% of parameters at full precision** to handle outlier values, improving performance.

### 2. Model Distillation

Trains a **smaller student model** to mimic a larger one, retaining **90-99% of the original performance** while running significantly faster.

- **DistilBERT:** 40% smaller than BERT, 60% faster.
  - **TinyBERT:** 7.5x smaller than BERT.
  - **MiniLM:** 50% smaller than BERT, retaining **99% of its performance**.
  - **DistilGPT-2:** A distilled version of GPT-2 for faster inference.
- 

## Final Takeaways

- **Fine-tuning** is necessary for optimal performance.

- **PEFT methods** (LoRA, Adapters, BitFit) drastically reduce computational costs.
- **Quantization and distillation** enable large models to run efficiently on smaller devices with minimal accuracy loss.

These techniques make **LLMs more practical for real-world deployment**, ensuring strong performance while reducing resource consumption.

---

## C6. LLM: Introduction to Software Engineering

### Software and the Economy

Software plays a fundamental role in modern economies, controlling critical systems across industries. Software expenditure constitutes a significant share of GDP, emphasizing its importance in both business and infrastructure.

### Key Concepts in Software Engineering

#### What is Software?

Software is a collection of:

- **Computer programs** that execute tasks.
- **Procedures & rules** governing operations.
- **Documentation** such as requirements, test plans, and user manuals.
- **Data** including project plans and deployment scripts.

#### What is Software Engineering?

The **multi-person construction of multi-version software**, involving:

- **Communication & coordination** challenges.
- **Long-term maintenance** considerations.

#### Types of Software

- **Embedded software** (e.g., software in cars, appliances).

- **Standalone applications** (e.g., office suites, social networks).
- **Enterprise systems** (e.g., banking and retail management).

## Software Criticality

- **Safety-Critical:** Failures may cause harm (e.g., self-driving cars).
  - **Mission-Critical:** Failures disrupt business operations (e.g., banking systems).
  - **Other Levels:** Varying degrees of importance based on impact.
- 

## Software Engineering Process

### Classical vs. Software Engineering

- **Classical Engineering:** Design → Manufacture → Maintenance.
- **Software Engineering:** Software Process → Deployment → Evolution.

## Software Properties

- **Functional Properties:** Core system functionalities.
- **Non-Functional Properties (ISO 25010):**
  - **Usability:** Ease of learning and satisfaction.
  - **Efficiency:** CPU, memory, and response time optimization.
  - **Reliability & Availability:** System uptime and defect rates.
  - **Maintainability:** Ease of modification.
  - **Security:** Protection against unauthorized access.
  - **Safety:** Prevention of harm.
  - **Dependability** = **Safety + Security + Reliability.**

## Software Development Phases

1. **Requirements Engineering** – Define system needs.
2. **Design** – Define system architecture.
3. **Coding** – Implement functionality.

- 4. **Testing** – Validate correctness.

## Software Engineering Laws

- **Fixing design errors is costly** – Early detection is crucial.
  - **Modularity & reuse** improve quality and reduce costs.
  - **Testing reveals defects but cannot prove their absence.**
  - **Software evolves over time**, requiring continuous adaptation.
  - **Processes must be adapted** to specific project needs.
- 

## Software Development Models

### 1. Waterfall Model

A **linear, sequential** approach.

- **Pros:** Well-structured, clear documentation.
- **Cons:** Rigid, testing happens late, difficult to adapt to change.

### 2. V-Model

Emphasizes **verification & validation** at every phase.

- **Pros:** High-quality assurance, strong control.
- **Cons:** Costly, requires thorough planning upfront.

### 3. Iterative Model

Builds software in **small increments**.

- **Pros:** More flexible, accommodates evolving requirements.
  - **Cons:** Requires a clear system vision before iteration.
- 

## Beyond Code Development

Software engineering extends beyond initial development:

- **Operation:** Deployment and execution.

- **Maintenance:** Continuous updates and bug fixes.
- **End-of-life:** Managing software retirement.

## Challenges in Maintenance

- **Architecture erosion:** Degrading system structure over time.
  - **Requirement creep:** Uncontrolled expansion of features.
  - **Refactoring:** Improving internal structure without altering behavior.
- 

## Agile Software Engineering

### The Agile Manifesto (Beck et al., 2001)

Emphasizes:

- **Customer satisfaction** and continuous delivery.
- **Self-organizing teams** with face-to-face communication.
- **Technical excellence** and simplicity.

## SCRUM Methodology

Roles:

- **Product Owner** – Defines priorities.
- **Scrum Master** – Facilitates progress.
- **Development Team** – Delivers working software.

Processes:

- **Sprint (1-4 weeks)** – Develops a functional increment.
- **Daily Standups** – Synchronizes team progress.
- **Sprint Review** – Demonstrates completed work.
- **Retrospective** – Identifies process improvements.

## Key SCRUM Concepts

- **Product Backlog:** List of all required features.

- **Sprint Backlog:** Tasks selected for the sprint.
  - **INVEST Principle** (Good backlog items are):
    - **Independent, Negotiable, Valuable, Estimable, Small, Testable.**
- 

## Final Takeaways

- **Balancing process discipline with adaptability is key.**
  - **Understanding requirements and designing for change** prevents costly revisions.
  - **Agile methodologies like SCRUM** enhance responsiveness and collaboration.
  - **Maintenance is continuous, not an afterthought.**
- 

# C7. LLM for Software Engineering (LLM4SE)

## Definitions

- **AI4SE:** Applying AI techniques to support software engineering.
  - **SE4AI:** Using software engineering practices to develop AI systems.
  - **LLMSE:** Integration of Large Language Models (LLMs) into software engineering.
  - **Assured LLMSE:** Ensuring reliable and verifiable LLM outputs in software engineering.
- 

## Current State of LLMs in Software Engineering

- Over **70 LLMs** have been applied to SE tasks.
- Three categories of LLMs:
  - **Encoder-only (e.g., BERT)** → Best for understanding.
  - **Encoder-decoder (e.g., T5)** → Best for understanding + generation.
  - **Decoder-only (e.g., GPT-4, Codex)** → Best for text/code generation.
- **Decoder-only models** are the most widely used in software engineering.

---

## Selection and Fine-Tuning of LLMs for SE

### Criteria for Choosing an LLM

- **Understanding code context** (e.g., variable scopes, dependencies).
- **Generating relevant content** (e.g., function implementations).
- **Responsiveness to fine-tuning** (e.g., adapting to specific tasks).
- **Performance on SE benchmarks** (e.g., correctness, efficiency).

### Fine-Tuning Techniques

- **Full Fine-Tuning** → Expensive but highly effective.
- **Parameter-Efficient Fine-Tuning (PEFT)** → Optimized adaptation:
  - **LoRA (Low-Rank Adaptation)**
  - **Prompt Tuning**
  - **Prefix Tuning**
  - **Adapter Tuning**
- **Reinforcement Learning (RLHF)** → Aligns models with human preferences.

---

## Datasets for LLM4SE

### Sources of Data

- **Open-source datasets** (e.g., GitHub, Hugging Face).
- **Collected datasets** (e.g., forums, blogs, documentation).
- **Constructed datasets** (e.g., synthetic data generation).
- **Industrial datasets** (e.g., proprietary business data).

### Types of Data

- **Text-based** → Programming problems, bug reports.
- **Code-based** → Open-source repositories, patches.

- **Graph-based** → UI representations (e.g., screenshots).
  - **Software repository data** → Commit histories, issues.
  - **Combined datasets** → Mix of source code, documentation, and test cases.
- 

## Applications of LLMs in SE

### 1. Requirements Engineering

- **Requirements Elicitation** → Generating Software Requirement Specifications (SRS).
- **Requirements Classification** → Categorizing functional vs. non-functional requirements.

### 2. Software Design

- **GUI Retrieval** → Using LLMs to generate UI prototypes.

### 3. Development

- **Code Generation** → Converting text into code (e.g., Codex, CodeGen, GPT-4).
- **Code Completion** → Assistive tools (e.g., GitHub Copilot, CodeGPT).
- **Code Summarization** → Automating documentation of code.

### 4. Quality Assurance

- **Bug Localization** → Identifying problematic code areas.
- **Test Generation** → Automating test case creation for better coverage.

### 5. Software Maintenance

- **Code Review** → Detecting errors and improving maintainability.
  - **Automated Refactoring** → Improving code structure over time.
- 

## Challenges & Opportunities

### Key Issues

- **Model Size & Deployment** → Large LLMs require high computational power.
- **Data Dependency** → Performance is tied to dataset quality.
- **Code Ambiguity** → LLMs may generate incorrect or insecure code.
- **Generalizability** → Performance varies across projects and domains.
- **Trust & Ethics** → LLMs can introduce bias, lack transparency, and pose security risks.

## Evaluation Metrics

- **Classification Tasks** → Precision, F1-Score.
  - **Recommendation Tasks** → Mean Reciprocal Rank (MRR).
  - **Generation Tasks** → BLEU, Pass@k.
- 

## Final Takeaways

- **LLMs significantly enhance** software engineering productivity and automation.
  - **Selecting the right model, dataset, and fine-tuning method** is crucial.
  - **Validation, monitoring, and ethical considerations** are necessary for reliability.
- 

# C8. Prompt Engineering

## What is Prompt Engineering?

- **Definition:** The practice of designing and optimizing prompts to effectively use Large Language Models (LLMs) without modifying the model itself.
  - **Alternative to fine-tuning:** Achieves adaptability without retraining the model.
- 

## Key Principles of Effective Prompt Engineering

1. **Specificness** → More precise prompts yield more focused outputs.

2. **Step-wise Approach** → Break down complex tasks into smaller steps.
3. **Iteration & Improvement** → Refine prompts through trial and error.

## What Makes a Good Prompt?

- ✓ **Clear and concise language**
  - ✓ **Assigning a role or persona to the model**
  - ✓ **Providing examples and context**
  - ✓ **Specifying output format**
  - ✓ **Continuous refinement**
- 

## Types of Prompt Engineering Techniques

### Basic Prompting Techniques

- **Priming** → Setting context before the task (e.g., describing a setting before generating a story).
- **Tabular Format Prompting** → Structuring outputs into tables for better readability.
- **Fill-in-the-Blank Prompting** → Allowing user choices within a structured prompt.
- **Perspective Prompting** → Generating responses from multiple viewpoints.
- **RGC Framework (Role, Goal, Context, Constraint)** → Defining the model's persona, objectives, and constraints.

### Shot-Based Prompting

- **Zero-Shot** → No prior examples; model relies on pretraining.
  - **One-Shot** → A single example to guide the model.
  - **Few-Shot** → Multiple examples to enhance performance and reduce bias.
- 

## Advanced Prompting Techniques

## 1. Chain of Thought (CoT) Prompting

- Introduced by **Wei et al., 2022** to enhance reasoning capabilities.
- Encourages **step-by-step breakdown** of complex problems.
- Useful in **math problems, logical reasoning, and multi-step processes**.

## 2. Self-Consistency with CoT

- **Wang et al., 2022**: Generates **multiple reasoning chains** and selects the most consistent final answer.

## 3. Tree of Thoughts (ToT)

- **Yao et al., 2023**: Extends CoT by structuring reasoning as a **tree**.
- Uses search algorithms (**BFS, DFS**) to explore solution paths.

## 4. Structured Chain of Thought (SCoT)

- **Li et al., 2024**: Applies CoT principles specifically to **code generation** by mimicking structured programming.
- 

## Reducing LLM Hallucinations

### 1. Retrieval-Augmented Generation (RAG)

- **Lewis et al., 2020**: Models **retrieve external knowledge** before generating responses.
- Solves issues like:
  - **Hallucinations** (false facts).
  - **Outdated responses**.
  - **Dependability of sources**.

### 2. ReAct (Reasoning + Acting)

- **Interleaves reasoning steps with external information retrieval**.
  - Enhances model reliability for **knowledge-intensive tasks**.
-

## Security Risks in Prompt Engineering

- ▲ **Prompt Injection** → Manipulating model behavior through adversarial prompts.
  - ▲ **Prompt Leaking** → Accidental exposure of sensitive data in model outputs.
  - ▲ **Jailbreaking** → Bypassing safety restrictions via crafted prompts.
- 

## Final Takeaways

- ✓ **Prompt engineering is a powerful alternative to fine-tuning.**
  - ✓ **Effective prompts improve accuracy, coherence, and task-specific performance.**
  - ✓ **Advanced techniques like CoT, ToT, and RAG enhance reasoning and reduce hallucinations.**
  - ✓ **Security risks must be mitigated through robust safeguards.**
- 

# C9. Agent Architectures in Large Language Models (LLMs)

---

## 1. Agentic Behavior in LLMs

### Traditional vs. Agentic LLMs

- **Traditional LLM applications** → Follow a **fixed workflow** (e.g., RAG retrieves documents before querying an LLM).
- **Agentic LLMs** → Decide their own control flow, choosing **steps, tools, and when to terminate a task**.

### Benefits of Agentic LLMs

- ✓ **Enhanced problem-solving** → More flexible task execution.
  - ✓ **Dynamic control flow** → Adjusts workflows based on the task.
  - ✓ **Increased efficiency** → Avoids unnecessary computations.
-

## 2. LLM Chains & Architectures

### LLM Chains

- **Definition** → A structured sequence of prompts guiding an LLM through complex tasks.
- **Example** → Summarizing an article in steps (**identify key points** → **generate summary** → **refine output**).

### Types of Chains

1. **Sequential Chain** → Linear steps where output of one step becomes input for the next.
  2. **Tree Chain** → Handles multiple inputs/outputs, allowing branching workflows.
  3. **Router Chain** → Routes tasks to specialized subchains for efficient processing.
- 

## 3. From Chains to Agentic Systems

- **Traditional AI** → Reacts to user input (e.g., answering a weather query).
  - **Agentic AI** → Takes **initiative, plans, and adapts dynamically** (e.g., providing a weather forecast + **suggesting clothes & activities**).
- 

## 4. Core Components of Agentic Architectures

1. **Agent** → The decision-making entity that orchestrates tasks.
2. **Tools** → Extend the agent's abilities (e.g., Calculator, Code Interpreter, Search API).
3. **Memory**
  - **Short-term memory** → Stores recent interactions.
  - **Long-term memory** → Retains knowledge over time (e.g., user preferences).
4. **Planning** → Breaks tasks into subgoals and determines execution order.

5. **Reflection & Self-Criticism** → Evaluates past actions to improve performance and reasoning.
- 

## 5. Example of an Agent in Action

**Task:** Plan a two-day AI & Robotics workshop.

- ✓ **Decomposition** → Agent breaks down the task (**venue booking, scheduling, budgeting**).
  - ✓ **Execution** → Each subgoal is processed sequentially (**find venues → shortlist → compare costs → book venue**).
  - ✓ **Reflection** → Agent evaluates past decisions ("**Was the venue cost-effective?**").
  - ✓ **Self-Criticism** → Identifies inefficiencies ("**Too much lecture time, add more Q&A sessions**").
  - ✓ **Final Execution** → Books venue, sends invitations, prepares materials.
- 

## 6. Memory & Knowledge Utilization

### Memory Storage Formats

- **Text** → Notes, documentation.
- **Images** → UI screenshots, diagrams.
- **Audio** → Speech recognition.
- **Structured Databases** → Organized data retrieval.

### Memory Retrieval

- ✓ Uses **filtering mechanisms** to recall relevant past data.

### Knowledge Sources

- **Textual** → Books, research papers, documentation.
- **Visual** → Image-based reasoning (e.g., analyzing UI screenshots).
- **Audio** → Speech processing.

- **External Data** → APIs, web scraping, structured databases (e.g., PubMed, Google Knowledge Graph).
- 

## 7. Advanced Reasoning & Planning Techniques

1. **One-Step Reasoning** → Single-pass planning (**fixed sequence of subgoals**).
  2. **Multi-Step Reasoning** → Iterative approach (**refines plans through multiple cycles**).
- 

## 8. Multi-Agent Interaction Models

### Types of Agent Interactions

- ✓ **Cooperative Agents** → Work together toward a shared goal.
  - ✓ **Adversarial Agents** → Compete against each other (e.g., AI evaluations).
  - ✓ **Mixed Agents:**
    - **Parallel** → Work independently but share information.
    - **Hierarchical** → Parent agents assign tasks to child agents.
- 

## 9. Reducing Hallucinations & Biases

- ✓ **Retrieval-Augmented Generation (RAG)** → Pulls **external facts** to reduce misinformation.
  - ✓ **Web Scraping & APIs** → Fetch **real-time data** for accuracy.
  - ✓ **Fact-Checking Mechanisms** → Reduce **biases and hallucinations**.
- 

## Final Takeaways

- ✓ LLMs can evolve from static tools to autonomous agents.
- ✓ Agentic architectures enable dynamic decision-making and tool use.
- ✓ Memory, planning, and reflection enhance adaptability and efficiency.

- ✓ Multi-agent interactions improve collaboration, competition, and real-world applications.
- 

## C10. Evaluation of Large Language Models in Software Engineering (LLMSE)

---

### 1. Evaluating LLM Output

#### Key Factors for Evaluation:

- **Deliverable Type:** Free text, structured text, code, test cases, etc.
  - **Software Engineering Phase:** Requirements, design, development, testing.
  - **Ground-truth Availability:** Whether a correct reference exists for comparison.
- 

### 2. Evaluating Requirements Extraction

#### Metrics:

- **Precision:** Measures how many extracted requirements are correct.

$$Precision = \frac{TP}{TP + FP}$$

- **Recall:** Measures how many total correct requirements were extracted.

$$Recall = \frac{TP}{TP + FN}$$

- **F1 Score:** Balances precision and recall.

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

#### Challenges:

- Different phrasing (e.g., "view trail descriptions" vs. "browse hiking trails").

## Solutions:

- **Synonym Lists, Word Embeddings, Sentence Embeddings** (e.g., BERT, SBERT).
  - **Example Metric:** Cosine similarity measures semantic closeness.
- 

## 3. Evaluating Design

LLMs assist in software design by generating:

- **Class Diagrams:** System structure.
- **Sequence Diagrams:** Message flows.
- **Use Case Diagrams:** User interactions.

### Structural Metrics:

- **Missing Dependencies:** Ensuring all necessary connections exist.
  - **Dependency Violations:** Avoiding cyclic dependencies or misplaced relationships.
- 

## 4. Evaluating Code Generation

### Functional Correctness:

- **Pass Rate:**

$$PassRate = \frac{PassedTestCases}{TotalTestCases} * 100$$

- Example: Testing a factorial function with different inputs (positive, edge, negative cases).

### Static Code Quality Metrics:

#### 1. Cyclomatic Complexity (CC):

$$CC = E - N + 2P$$

- Low CC (1-10): Simple, maintainable code.
- High CC (50+): Complex, needs refactoring.

## 2. Maintainability Index (MI):

- Score from 0-100 (higher is better).
  - Based on complexity, lines of code, and readability.
- 

## 5. Evaluating Runtime Performance

- **Execution time:** Time required for execution.
  - **Throughput:** Number of operations per time unit.
  - **Memory consumption:** Tracks RAM usage.
  - **CPU time used:** Measures processor load.
  - **Error rate:** Frequency of failures during execution.
- 

## 6. Code-Specific Similarity Metrics

### Challenges with BLEU for Code Evaluation:

- Small vocabularies.
- Tree-structured syntax.
- Precise semantics.

### CodeBLEU (Ren et al., 2021)

- **Improvements over BLEU:**
    - **AST Matching:** Abstract Syntax Trees.
    - **Data-Flow Matching:** Variable usage and ordering.
- 

## 7. Evaluating Test Case Generation

### Test Coverage Types:

- **Line Coverage:** Percentage of executed lines.
- **Branch Coverage:** Ensures all conditional branches are tested.
- **Function Coverage:** Checks if all functions are called.
- **Path Coverage:** Tests all possible execution paths.

## Execution Success Rate

$$ExecutionSuccessRate = \frac{PassingTestCases}{TotalTestCases} * 100$$

## Regression Testing:

Ensures new updates don't break existing functionality.

## Mutation Analysis:

- Introduces small mutations (code changes) and evaluates if test cases detect them.
- **Mutation Score:**

$$MutationScore = \frac{KilledMutants}{TotalMutants} * 100$$

---

## 8. Identifying Flaky Tests

### Flaky Tests:

Tests that inconsistently pass or fail without code changes.

### Causes of Flakiness:

- External dependencies (e.g., APIs, databases).
- Timing issues (e.g., race conditions).
- Shared state (e.g., global variables).

### Metrics:

- **Flakiness Rate:**

$$FlakinessRate = \frac{Failures}{TotalTestsRuns} * 100$$

- **Stability Index:**

$$StabilityIndex = 1 - FlakinessRate$$

---

## 9. Feedback-Based Evaluation

- **Blind Peer Review:** Reviewers compare outputs without knowing the source.
- **Real-World Testing:** Deploying generated code to assess stability and performance.
- **Readability & Maintainability Reviews:**
  - Naming conventions, comments, modularity.
  - Example Checklist: **Is the test case clear? Does it cover positive & negative cases?**

---

## Final Takeaways

- LLMs require **rigorous evaluation** in software engineering tasks.
- Precision, recall, and cosine similarity help assess **requirement extraction**.
- Code correctness, complexity, and performance must be measured systematically.
- **Mutation testing** and flaky test identification ensure **robust test suites**.
- **Feedback from real-world use cases** enhances evaluation accuracy.